20260418 - Class 11 - JS Continued

18 April 2026 20:37

* Quick Recap:

- Callback functions # Asynchronous Behavior
- XMLHTTP Request → xhr requests → callback queue
- Fetch → promise → microtask queue.
- Web APIs.
- promises → .then, .catch, .finally.
- promise chains
- Asyn/await → sugar coating for promises.
- promis.all/race/any → concurrent behavior.
- Console.error( )

* P1 = Promise 1 ....
P2 = Promise 2 ....
P3 = Promise 3 ....

promiseArr = [P1 , P2 , P3]

res = Promise .all (promiseArr) →
" .race (first promise gets some state (resolve/reject))
res = " .any (first promise gets resolve)

Higher Order Functions in Array:

↳ map

- map
- filter
- reduce

```
const inputs = ['1', '2', '3', '4'];
console.log(inputs);

const numbers = inputs.map(input => {
    return Number(input);
})

console.log(numbers);
```

```
(4) ['1', '2', '3', '4']
(4) [1, 2, 3, 4]
ctrl i to turn on code suggestions
```

```

const evens = numbers.filter((number) => {
    return number % 2 == 0; // true / false
})

console.log(evens);
```

F T F T

```
(4) ['1', '2', '3', '4']
(2) [2, 4]
ctrl i to turn on code suggestions
```

Functional Programing / Procedural / OOPS

pure functions

functions which return same o/p for same I/p.

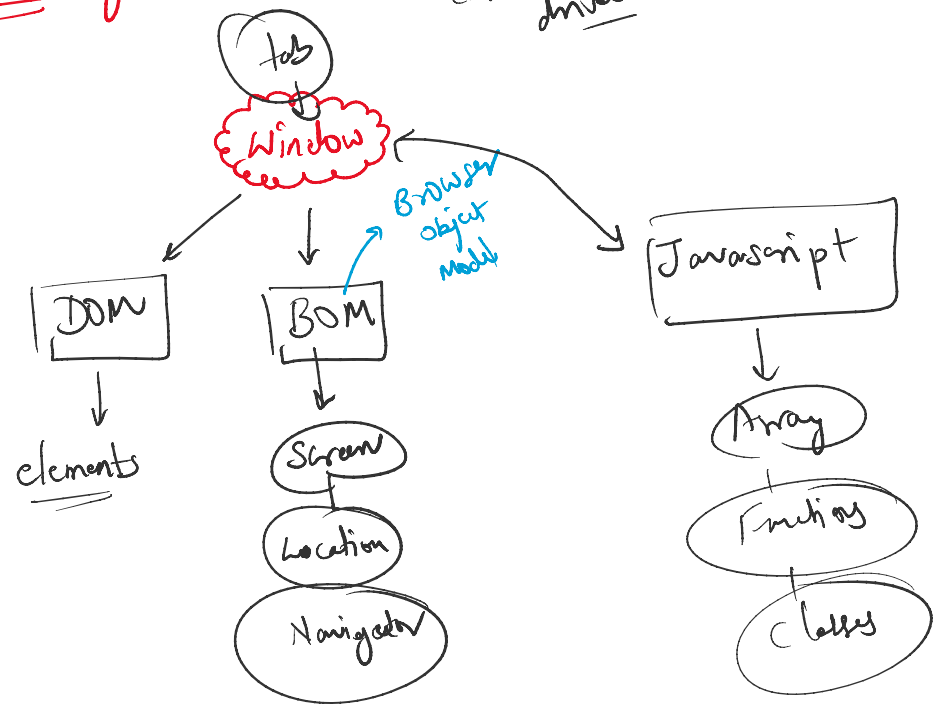
Step by-step

Assembly

Imp.

* Web Development Using JS

chromium driven

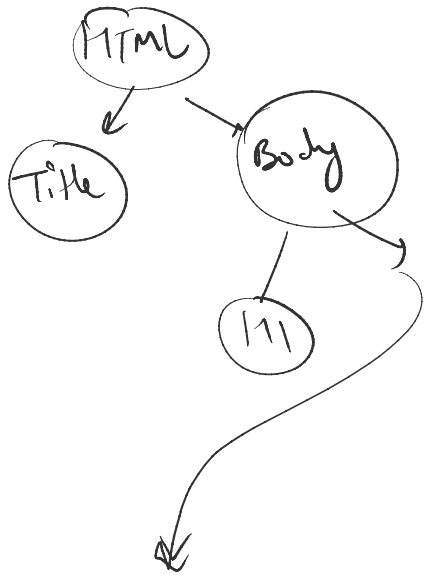


DOM

<HTML>

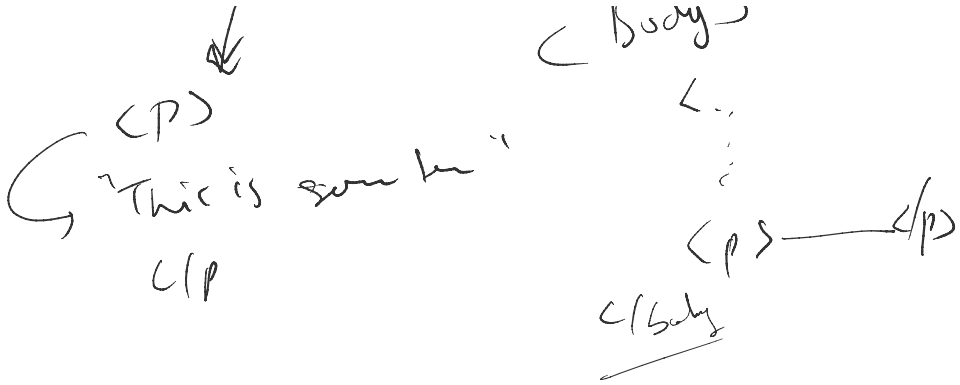
<Body>

DOM



</HTM>

<Body>



"This is some text"
</p
<Body>
<..
 ——
</body>